GPX Overlay Studio: Client-Side Rendering Architecture

Final plan: CSS/DOM overlay renderer with lossless capture, selectable client or server export, exact HDR tone mapping

---

2026-10-13

Prepared for Liria / fhu

Architecture only - no implementation in this document

Revision 2 - incorporates review decisions of 2026-10-13

Contents

1 Executive Summary **3**

2 Review Decisions Incorporated **4**

3 Phone Support Analysis **5**

4 Current Architecture **6**

5 Target Architecture **7**

5.1 Component responsibilities 7

6 Export Mode A: Fully Client-Side **9**

7 Export Mode B: The PNG Bridge **10**

8 Migration Phases **11**

9 Risks and Mitigations **12**

10 Alternatives Considered **13**

11 Closed Decisions (2026-10-13) **14**

12 CSS Effect Palette **15**

13 Open Items **16**

14 Rough Effort Estimate **17**

15 Summary **18**

1 Executive Summary

GPX Overlay Studio renders telemetry overlays today on the server: a Python compositor draws every widget per frame, and ffmpeg burns them into the video. This document is the final architecture for moving rendering into the browser. It incorporates three review decisions: **CSS effects matter**, **the overlay must be captured losslessly**, and **client-side rendering is a requirement**, with the export backend selectable between client and server.

The architecture in one paragraph. The **CSS/DOM overlay layer of the editor becomes the renderer** - the same layer the user sees is the layer that is captured, so preview and export are identical by construction and every CSS effect (text shadows, blur, gradients, easing) is available for free. For export, each frame time is set into the model deterministically and the overlay is rasterized in-tab to **lossless RGBA with alpha**. Two export modes consume that capture through one renderer: **Mode A, fully client-side** - source video decoded in the browser, composited with the overlay in one pass with exact tone mapping, hardware-encoded, muxed in JS, downloaded directly; and **Mode B, the PNG bridge** - the same lossless frames leave the browser as a PNG sequence and one ffmpeg pass finishes the job server-side. The UI offers **Render on: Auto / This device / Server**, with capability detection choosing sensibly and the user free to override. Tone mapping is the exact tuned HLG chain, implemented as a WebGL2 fragment shader so it runs on phones as well as desktops.

This replaces the earlier WebGPU-first draft. WebGPU survives only as an optional accelerator; it is on no critical path. The GPU work that draft required (SDF gauges, glyph atlases) is deleted - CSS draws the widgets.

2 Review Decisions Incorporated

These are the three constraints from review, and where each lands:

- **CSS effects are a plus.** The renderer is the DOM overlay itself. No shader reimplementation of widgets exists anywhere in the plan.
- **Lossless overlay.** No lossy alpha-video intermediate exists in either mode. Mode A blends the overlay once at full precision; Mode B transports PNG frames. Golden-frame tests pin this.
- **Client-side rendering is a must.** Mode A is end-to-end in the browser. Mode B keeps all rendering client-side too; the server runs a single ffmpeg filter with no per-frame drawing. The user selects which mode to use.
- **Exact tone mapping.** The tuned HLG-to-SDR chain is ported one-to-one into a WebGL2 fragment shader with the same constants, verified against server output by PSNR golden tests. No approximation grade.
- **Phone use is expected.** Upload and render may happen on a phone; see the support analysis below. The design does not assume desktop.

3 Phone Support Analysis

The review question was whether a phone can do this work. Checked against browser support as of October 2026:

- **WebCodecs (the capability that matters for Mode A):** the video classes - VideoDecoder, VideoEncoder, VideoFrame - have been in iPhone Safari since iOS 16.4 (partial implementation covering exactly these interfaces) and are complete in Safari 26 across macOS, iOS, and iPadOS. Android Chrome has shipped WebCodecs since 2023. Hardware H.264 encode is available on modern phones. **Mode A is viable on an up-to-date phone.**
- **WebGL2 (the capability the exact tone map needs):** universal on phones of the last five years - iOS Safari since iOS 15, Android everywhere. No concern.
- **WebGPU (optional accelerator only):** Android Chrome since 113 (2023); on iPhone, behind experimental flags in the iOS 18 era and full in iOS 26 Safari. Because the renderer is CSS and the tone map is WebGL2, **nothing in the plan requires WebGPU**. A phone without it simply skips the optional acceleration layer.

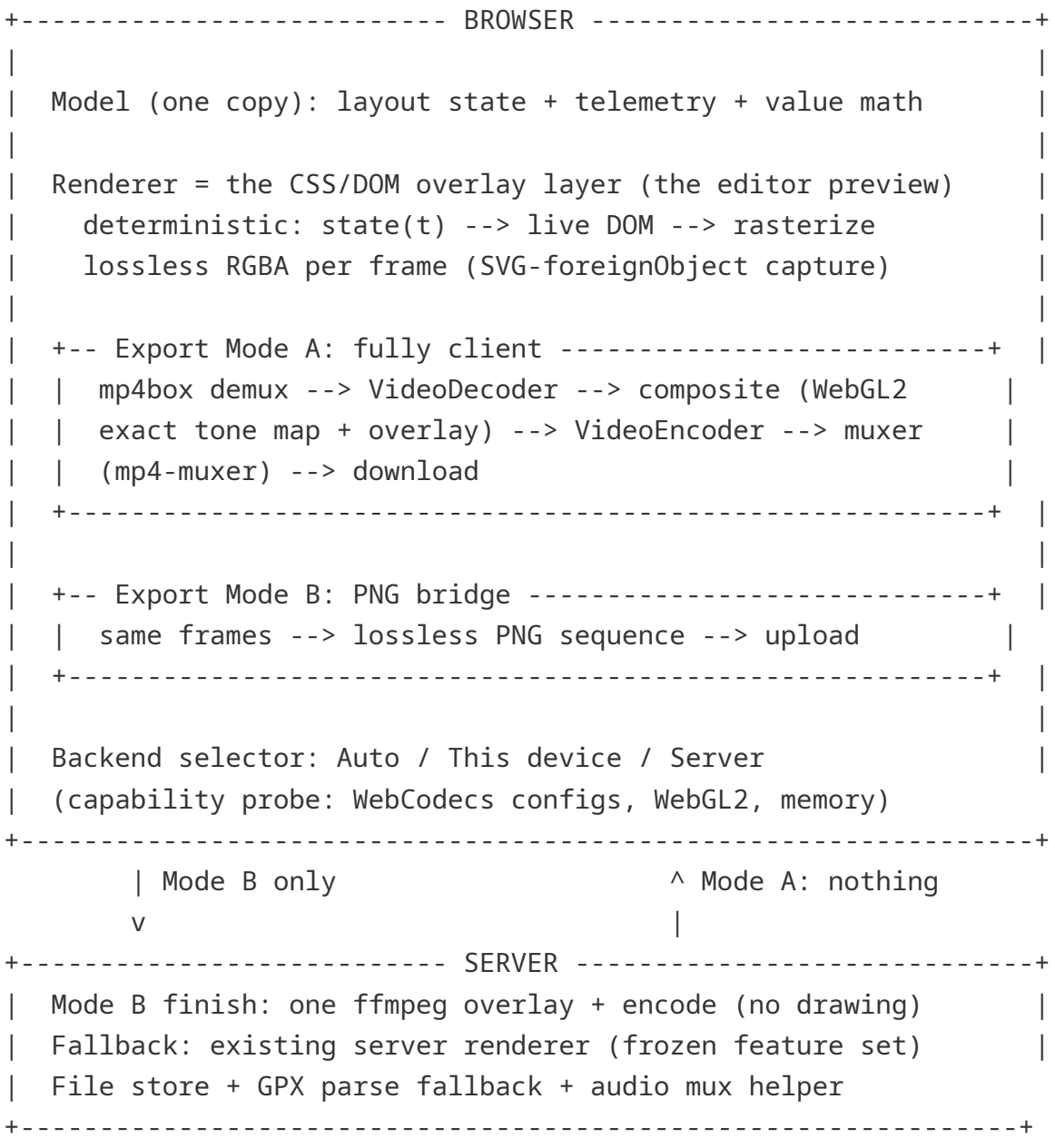
Conclusion: the phone worry dissolves. The one real capability gate is WebCodecs for fully-client export, and it is present on current phones; where it is missing or its encoder probe fails, the server path is one tap away - which is exactly why the backend selector exists.

4 Current Architecture

The system as it stands after the 2026-10-13 iteration:

- **Ingest** (`app.py`): video saved as uploaded; probe records rotation-aware display dimensions, HDR flag, size, bitrate; GPX normalized and parsed into `telemetry.json`.
- **Model**: layout state JSON in the browser (widgets, styles, themes, geometry, appearance flags) plus telemetry points. Value math (`sampleAt`, `fmtVal`, `rangeFor`) exists in JavaScript for the preview and mirrored in Python for rendering - the duplication this plan deletes.
- **Render** (`overlay.py` + ffmpeg): Pillow draws widgets per frame on the server; ffmpeg overlays and encodes at CRF 15 with in-encode HDR tone mapping.
- **Preview**: a `<video>` under DOM mockups, plus a server-rendered pixel preview endpoint.

5 Target Architecture



5.1 Component responsibilities

Model layer. Unchanged in shape; gains exclusivity. The JS value math becomes the single definition and the Python mirror is retired (the server renderer, while it lives, is feature-frozen).

Renderer - the CSS/DOM overlay. The editor's overlay layer is promoted from preview to source of truth. Rendering a frame at time t means: set telemetry values into the DOM widgets (the same code that updates the scrub preview today), then rasterize. Effects, fonts, and layout are CSS and therefore identical between what is seen and what is exported.

Deterministic capture. Every frame is a pure function of time (the pdoom-video principle). CSS animations that run on wall-clock time are disallowed in the overlay; all motion comes from the model state at t. This is what makes preview and export identical.

Rasterizer. SVG-foreignObject serialization (the html2canvas family) rendered through the browser's own engine into an OffscreenCanvas - lossless RGBA with alpha per frame. Fidelity caveat tracked in risks: a handful of CSS effects (backdrop-filter, some blend modes) can rasterize slightly differently than live compositing; golden-frame tests police the gap.

Backend selector. On project load the app probes: WebCodecs `VideoEncoder.isConfigSupported` for the target codec, WebCodecs decode support for the source codec, WebGL2 availability, and a conservative memory estimate. Auto picks Mode A when the probe passes, Mode B otherwise; the user can override in either direction. The selector is honest: when a capability probe fails, that option is labeled as such rather than silently downgraded.

6 Export Mode A: Fully Client-Side

The target path. All work in the tab:

- **Demux and decode.** mp4box.js reads the MP4 progressively and feeds encoded chunks to `VideoDecoder`; frames come out as `VideoFrame`. Export decodes sequentially (frame-exact); preview seeks decode from the nearest keyframe and drop to the target time.
- **Composite.** Each decoded frame enters the composite pass with the rasterized overlay. The **exact tone map** lives here: the tuned chain (linearize, hable tone map, BT.709 matrix, contrast 1.32, saturation 1.45, brightness -0.09, gamma 0.93) as a WebGL2 fragment shader with identical constants, applied to HDR sources only. Golden-frame PSNR tests pin the output to the server pipeline's look.
- **Encode and mux.** `VideoEncoder` (hardware H.264 where supported; probe falls back to VP9/AV1) feeds mp4-muxer, which writes the MP4 with the source audio track. Fully offline result; the server sees nothing.

Render time is expected in the 20-40 s range for an 8-30 s clip at 1080x1920 on current hardware; the composite and tone map are GPU work, encode is hardware, rasterization is the floor. 4K works and is slower.

7 Export Mode B: The PNG Bridge

The same capture, transported:

- The client streams the lossless RGBA frames out as PNG (roughly 25-60 MB per clip at 1080x1920 - acceptable for occasional use).
- The server runs one ffmpeg overlay and encode - a filter job, not rendering: it draws nothing per frame.
- The PNG sequence doubles as a pixel-exact debug artifact and an archival format.

Mode B is also the v1 shipping path: it is days of work on machinery we already own, it satisfies client-side rendering (the server draws nothing), and it ships while Mode A's media plumbing lands behind the same selector.

8 Migration Phases

Phase 0 - Unify the model. Extract widget model and value math into one shared module; define the frame-at-time-t contract; freeze a golden test set from real projects (server-rendered reference frames plus per-widget value assertions). Exit: golden set committed and reproducible.

Phase 1 - CSS capture renderer. Promote the DOM overlay to the renderer; build deterministic state-setting and SVG-foreignObject rasterization; switch the editor preview to true frames (the pixel-preview button dies). Exit: captured frames match golden frames within tolerance, including the CSS effects we care about.

Phase 2 - Mode B export (v1 ships). PNG sequence out of the browser, one ffmpeg overlay and encode server-side, progress and cancellation. Exit: a full export from the browser equals the server render within PSNR tolerance and Liria signs off.

Phase 3 - Mode A export (fully client). mp4box demux, VideoDecoder, WebGL2 exact tone map, VideoEncoder, mp4-muxer, audio carry, plus the capability probe and selector UI. Exit: a phone and a desktop both produce a final MP4 with no server call; golden frames match.

Phase 4 - Optional WebGPU acceleration. Where available, move the composite and tone-map pass to WebGPU for speed and 4K headroom. No user visible change; measurable speedup. Exit: export wall-clock improves on supported hardware with identical output.

Phase 5 - Cutover and fallback. Selector polish, honest labeling, the old server renderer behind an explicit fallback button. Exit: default flow never touches the per-frame server path.

9 Risks and Mitigations

Risk	Impact	Mitigation
DOM rasterization fidelity (backdrop-filter, blend modes)	Preview looks right, export differs	Golden-frame tests per effect; restrict effect palette to rasterization-safe CSS; visual QA checklist
WebCodecs quirks on partial implementations (iOS 16.4-18.x)	Mode A flaky on some phones	Capability probe with real isConfigSupported calls; Mode B is always there; test on Liria's phone early (Phase 3)
Rasterization speed on older phones	Slow export	Mode B is rasterization-bound too but trivially parallel; progress UI; advise Mode B on weak devices; WebGPU/WebGL blits reduce copy cost
Tone-map shader drift from the server grade	Color changes between old and new outputs	Port constants one-to-one; PSNR golden tests against server-rendered frames
mp4-muxer / mp4box edge cases (odd codecs, variable frame rate)	Export fails	Phase 3 covers the real clip zoo (phone HDR Main 10, slo-mo, chat-compressed); server Mode B as escape hatch
Memory on phones (full-resolution frames in flight)	Tab crashes	Stream frames with bounded buffers; never hold the decode; cap concurrent canvases at 2
OSM tile policy or CORS change	Map widget breaks	Cache-first; tile proxy fallback on the server
Lossy intermediate creep (someone swaps PNG for WebM alpha)	Silent quality loss	Architecture rule: no lossy overlay intermediate; golden tests fail loudly if it appears

10 Alternatives Considered

Option | Verdict | Why |
- | | - |
WebGPU-first widget renderer (revision 1) | Superseded | CSS rendering achieves parity and effects at a fraction of the cost; WebGPU kept only as accelerator |
Headless-Chrome capture server-side (pdoom-video literal) | Rejected as primary | Violates the client-side rendering requirement; kept as mental model only |
Lossy alpha video intermediate (WebM/ProRes) | Rejected | Loses the lossless requirement for no benefit at our clip lengths |
ffmpeg.wasm whole export | Rejected | CPU-bound WASM, 5-20x slower, no hardware encode |
Server-only rendering (status quo) | Rejected as default | Retained as explicit fallback only |
Canvas2D widget port instead of CSS capture | Folded in as fallback of Phase 1 | If foreignObject fidelity fails on some effect, that effect renders via canvas; same interface |

11 Closed Decisions (2026-10-13)

- **Audio is required.** Fully client-side audio is part of the Phase 3 gate: demux, `AudioDecoder`/`AudioEncoder`, and mux carry the source audio track. Note the consequence: the WebCodecs *audio* classes are complete only from Safari 26 / current Android; older phones fall back to Mode B, where the server muxes audio in its single ffmpeg pass.
- **4K is not a requirement.** The WebGPU acceleration phase is parked as a future speed improvement only; nothing depends on it.
- **Effect palette: tiered (below).** Tier 1 effects are guaranteed; Tier 2 ships only what golden-frame tests prove pixel-clean; Tier 3 effects live in the composite shader.

12 CSS Effect Palette

The styling surface the widget system exposes, and how each tier is guaranteed:

Tier 1 - guaranteed identical between preview and export. Bundled fonts (Orbitron and friends) with size, weight, letter-spacing, small-caps, alignment; text colors; solid and alpha backgrounds; linear and radial gradients; pill and rounded-rectangle shapes with arbitrary borders; multi-layer text-shadow and box-shadow (outer and inner, blur and spread); opacity and alpha layering. These are the curated appearance-panel controls.

Tier 2 - offered behind golden-frame tests. Element blur; frosted glass (backdrop-filter) - the known rasterization risk, shipped only where tests show pixel-clean output, otherwise restricted; clip-path shapes and mask-image; blend modes between overlay widgets.

Tier 3 - composite-time shader effects. Overlay glow, vignette, film grain, and whole-overlay blend modes against the footage (screen, soft-light, dodge). These run in the export composite pass rather than in CSS, so they are exact and can also affect the video side of the blend.

Excluded by design. Wall-clock CSS animations and transitions (the deterministic-frame rule; value motion is model-driven, optionally using CSS easing curves as interpolation styles); non-bundled fonts; live external resources. An “advanced CSS” escape hatch exposes Tier 1 and tested Tier 2 properties with an allowlist.

13 Open Items

1. **None blocking.** All three review questions are answered above. The Phase 1 golden tests may still refine which Tier 2 effects ship; that is a palette detail, not an architecture change.

14 Rough Effort Estimate

Estimates are focused implementation time.

Phase	Scope	Estimate
Phase 0	Model unification, frame contract, golden tests	1-2 days
Phase 1	CSS capture renderer, true preview	2-3 days
Phase 2	Mode B export (v1 ships)	1-2 days
Phase 3	Mode A fully client export + selector	3-4 days
Phase 4	WebGPU acceleration (optional)	3-5 days
Phase 5	Cutover, fallback, polish	1-2 days
Total without Phase 4		**8-13 days**
Total with Phase 4		**11-18 days**

15 Summary

Render with CSS, capture losslessly, export client-side with an exact tone-mapped composite, and keep a selectable server path for weak devices. The preview is the renderer, so fidelity is structural; the lossless rule is enforced by design and by tests; phones are first-class (WebCodecs and WebGL2 cover them; WebGPU is optional). Mode B ships the v1 quickly on existing machinery, Mode A removes the server from the loop entirely, and the old server renderer remains as an explicit fallback. Eight to thirteen focused days to the complete non-WebGPU picture.